RMIT Classification: Trusted

RMIT UNIVERSITY

VC-268-EVENTCONNECT

TECHNICAL REPORT

Version 1.01

Start Date: **24 /07 /2023**

Authors : **Zain Nauman, s3909208**
Alli Toth, s3902333
Jed Beattie, s3840131
Paul Likiardopulos, s3843250
George Stergiadis, s3895662

RMIT Classification: Trusted

TABLE OF CONTENTS

1 EXECUTIVE SUMMARY....4

2 INTRODUCTION....5

3 REQUIREMENTS....6
Functional requirements specification....7
Non-functional requirements specification....8

4 ARCHITECTURE....9
Overview....9
Database....10
Continuous Integration Pipeline....12

5 TECHNICAL FRAMEWORK....13
Testing: 13
Backend:....13
Frontend:....13
Database:....13
Deployment solutions:....13
Version Control:....14

6 IMPLEMENTATION....15

7 DEPLOYMENT INSTRUCTIONS....17

8 TEST SPECIFICATIONS....18

9 TESTING RESULTS....23

10 CYBER SECURITY....24
Validation....24
Password Hashing....25
Securing endpoints and cloud deployment....25

11 IMPACT ON STAKEHOLDERS....26

RMIT Classification: Trusted

1 EXECUTIVE SUMMARY

This report delves into the pivotal role networking plays for startups, spin-offs, and innovative SMEs. It emphasizes the significance of events such as South Summit, 4YFN, and The Next Web in facilitating connections, accessing funding, and sharing knowledge. However, the challenge lies in selecting the right events amid a proliferation of options. To address this, Innovent has developed a mobile web application that streamlines event discovery and enrolment.

The application caters to essential requirements including user authentication, sign-up, event access, and profile management. Users can subscribe to events, receive relevant notifications, and select events based on specific tags. Non-functional requirements encompass response time, scalability, security, usability, and maintainability, ensuring a seamless user experience.

Architecturally, Innovent adopts a database-driven approach with an intermediary API layer facilitating data retrieval and manipulation. The frontend interfaces with this API to present information to users. The database schema was meticulously designed to accommodate complex relationships between users, events, and tags. Additionally, a Continuous Integration Pipeline was implemented to uphold code quality and integration testing.

The technical framework leverages technologies like Node.js, Express, MySQL, Jest, ESLint, React, Ionic, AWS and GitHub. This combination forms a robust foundation for development and deployment. The implementation centres around the Ionic framework, enabling users to log in, sign up, view events, and edit profiles. While some features are still in development, Innovent envisions a platform that helps event management.

In terms of deployment, Innovent advocates for cloud-based solutions using Amazon EC2 for backend and frontend, coupled with Amazon RDS for database management. Testing, conducted using Jest, has yielded successful outcomes, though some minor inconsistencies in status code responses were identified. Cybersecurity is of a high concern, and measures including function validation, token and model validation, password hashing, and endpoint security have been put in place to fortify the application against potential threats.

Innovent's initiative holds the potential to significantly augment networking opportunities for startups and SMEs, ultimately contributing to a more connected ecosystem. By facilitating tailored, inclusive gatherings and promoting global connections, Innovent aspires to create cultural exchange and diversity. The platform stands to build social interaction and community engagement.

RMIT Classification: Trusted

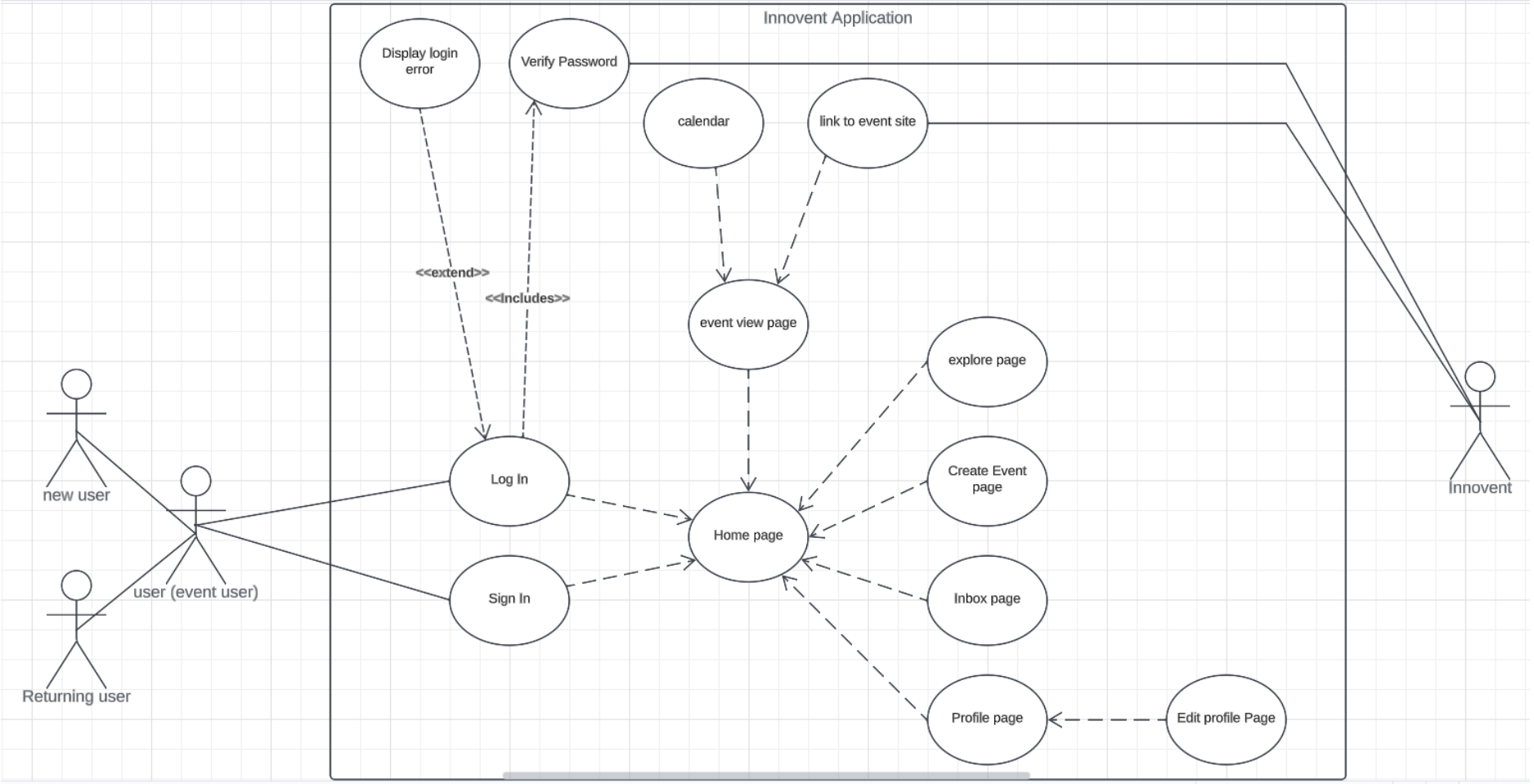
2 INTRODUCTION

Networking plays a crucial role in the startup, spin-offs, and innovative SMEs. Networking is essential in this realm, and how attending events like South Summit, 4YFN (4 Years from Now), The Next Web, and similar gatherings can enhance the performance of startups and base technology SMEs. These events are known for attracting a diverse range of industry professionals, investors, and innovation enthusiasts. Building connections: Expanding your professional network allows you to tap into valuable resources, gain insights from experienced individuals, and establish mutually beneficial relationships. Access to funding: Startups and SMEs often require external funding to fuel their growth. Knowledge sharing: Attending events allows you to learn from industry leaders, experts, and successful entrepreneurs through keynotes, panel discussions, and workshops. Collaboration opportunities: Networking events unite individuals with diverse skill sets and expertise. Market exposure: Startups and SMEs often struggle with limited visibility in the market. Networking events provide a platform to showcase your products, services, and innovations to a targeted audience. Feedback and validation: Networking events offer opportunities to present your ideas and receive feedback from industry professionals and potential customers. There are too many events yearly, and there are increasing even more that the challenge here is to decide what events and post-events will be the right for you and your company to attend according to your needs and what you are looking for.

Over the course of this semester, we have developed a mobile web application that connects prospective event goers with allows users to streamline their experience of discovering, researching, and enlisting into event that they would be interested in attending.

3 REQUIREMENTS

Figure 3.1 Use case diagram



RMIT Classification: Trusted

FUNCTIONAL REQUIREMENTS SPECIFICATION

Table 3.1 Functional requirements

Req. ID	Description	Priority	UC covered
1	The system must allow the user to log in by entering their email and password.	High	4
2	The system must allow the user to sign up using their email, a password, and a profile.	High	3,7
3	The system must give user access to event details.	High	16
4	The system must give user access to event website.	Low	16
5	The system must allow the user to add event details such as name, description, location and tags to create an event.	Low	8, 12
6	The system must allow a user to edit their profile information such as name, email and profile.	Medium	15
7	The system must allow a user to tap on tag buttons to select their tags.	High	1
8	The system must allow the user to subscribe to events/hosts and be notified when the host posts an event.	Low	2
9	The system must allow business users to receive notification relevant to their business.	Low	6
10	The system must allow user to pick events based on its tags.	Medium	13
11	The system must allow the addition of events to the calendar.	Low	14
12	The system must display relevant events on the homepage to a user.	Medium	5

RMIT Classification: Trusted

NON-FUNCTIONAL REQUIREMENTS SPECIFICATION

Table 3.2 Non-functional requirement

Category	Description
Response time	The system should respond to user queries within a maximum of 2 seconds under normal load conditions
Scalability	The application should be able to handle a minimum of 10,000 concurrent users without significant performance degradation
Throughput	The system should be able to process at least 1000 search queries per minute.
Availability	The application should be available 99.9% of the time, excluding planned maintenance
Availability	The system should have a built-in mechanism for failover and redundancy to ensure uninterrupted service.
Security	The application should implement strong encryption (SSL) for data transmission.
Security	User authentication and authorization mechanisms should be in place to protect user data and prevent unauthorized access.
Security	The system should comply with relevant data protection regulations
Usability	The user interface should be intuitive and user-friendly, ensuring that users can quickly and easily find events.
Usability	The application should support multiple languages and be accessible to users with disabilities.
Scalability	The system should be designed to handle a growing number of users and events without a significant drop in performance.
Maintainability	The codebase should follow best practices and be well-documented for ease of maintenance and future enhancements
Maintainability	Regular updates and patches should be easily deployable without causing service interruptions.

RMIT Classification: Trusted

4 ARCHITECTURE

OVERVIEW

To quickly summarize the architecture powering our project, we have a database which holds customer, event, and tag information and relationships between the data. This information is accessed using an intermediary layer through API calls which acts as a middleman between the database which is responsible for manipulating and retrieving relevant data based on the final layers requests, which is the frontend. The frontend is the layer that end users will see and interact with.

As seen in figure 4.1, during development we opted for locally running the database and API due to no hosted solution being provided by the client.

During our deployment phase (for client testing), parts of our architecture were migrated to the cloud, as shown in figure 4.2.

Figure 4.1 Simple overview of local architecture

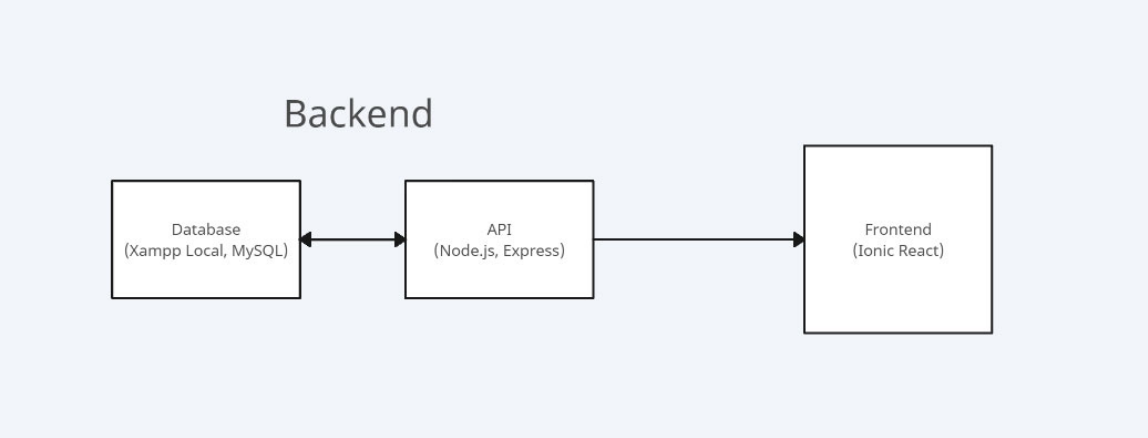
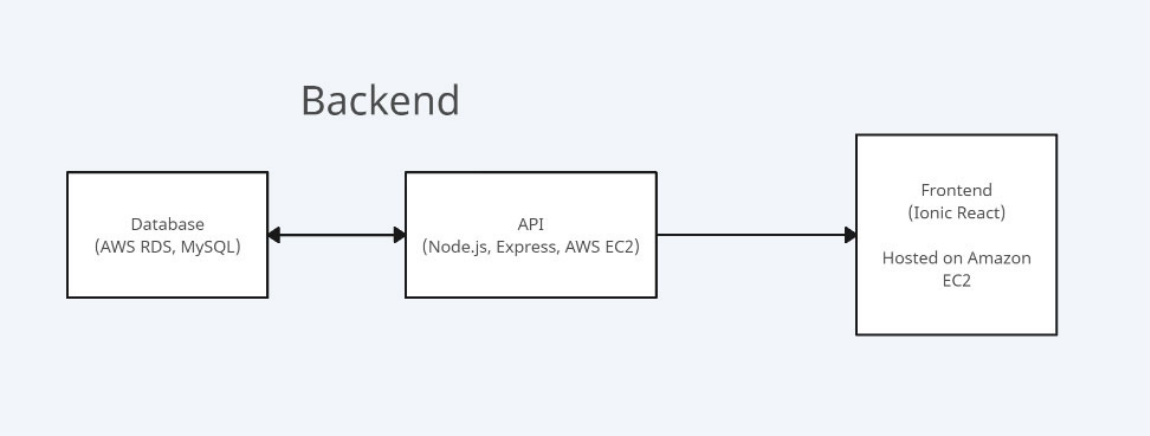


Figure 4.2 Simple overview of deployed architecture



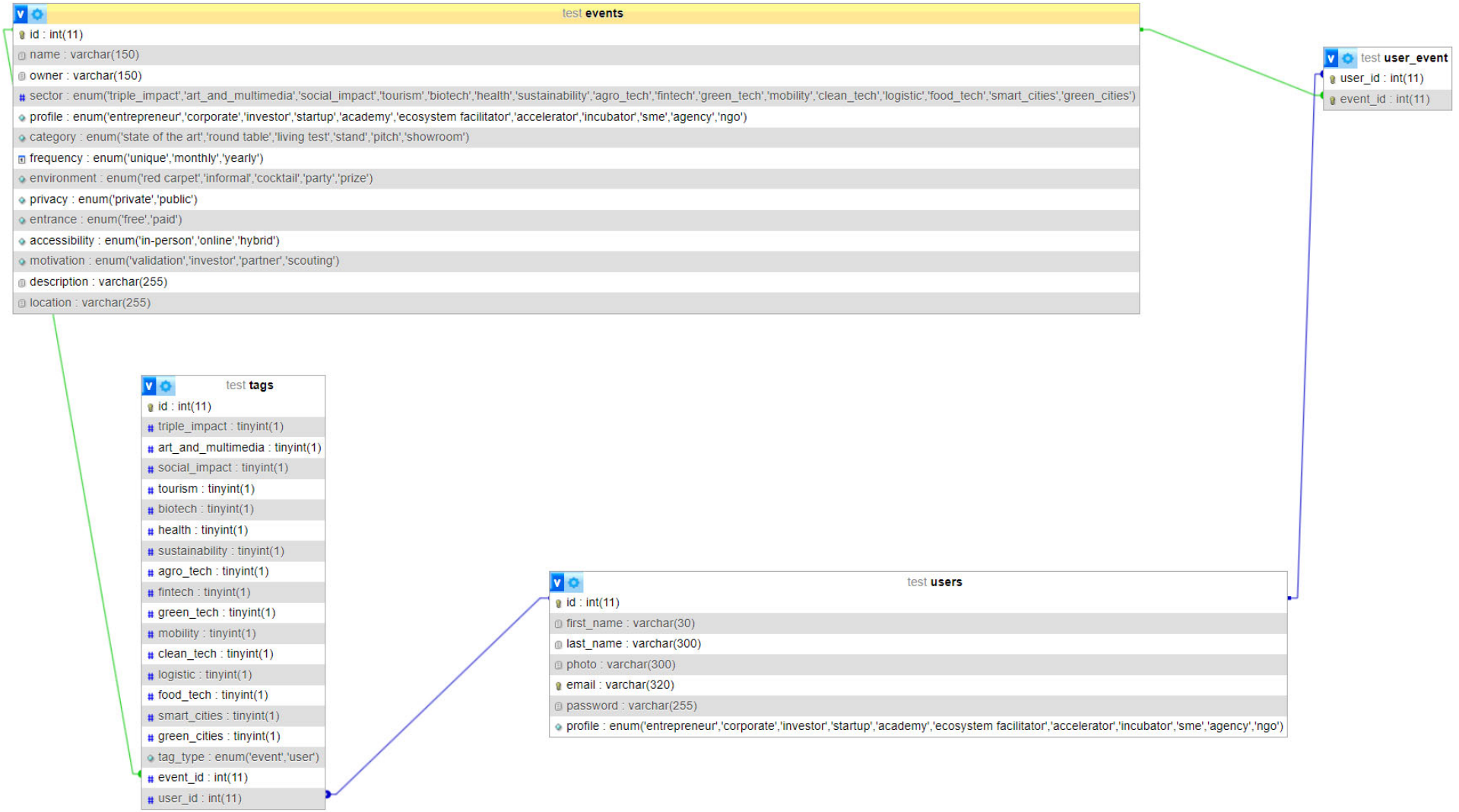
DATABASE

For this project we used the first sprint to discuss what our database would require in terms of flexibility, relationship between different data and how the client would expect the data to be presented and manipulated. The first point of consideration in our process was to determine whether we should opt for a traditional relation database or a NoSQL database like mongo db. From the requirements that were given to us by our client and inferences we had to make given the limited scope of the information handed to us, we determined that the different entities/tables in the database would need to have some form of explicit relationship as we expected there to be a lot of joining between entities. Due to this we decided to choose a relational database, more specifically PhpMyAdmin's MySQL local server db as NoSQL is quite inefficient when handling lots of join operations between documents/tables.

Our goal as backend developers/database modellers was to become familiar with what the client wanted the data to look like and the expected data that would need to be presented in the final app. Upon inquiring specific details from the client and creating mock-ups, we came to the realisation that relationship between users-tags and events-tags would be many to many which is something that is difficult to handle efficiently in relational databases. To circumvent this, we spent majority of our second sprint coming up with a database schemas that solved the issue in different ways and we came up with two equally calid solutions: use the JSON_CONTAINS keyword in MySQL that can parse through JSON strings which can be used to store lists (essentially a many to many relations); or make an intermediary table that to store which users will be attending which events. We opted for the second option as we realised that adding and removing users from an event would be ever expensive in terms of write operations, so parsing through a JSON string would be significantly less efficient that simply reading and writing to a single table (which can be done in constant time as opposed to linear). Another issue we had was to link tags to both users and events, as those were also many to many relationships. Our solution to that was to create a Boolean table where the columns represented the different tags and whether the tags corresponded to an event or user (as tags were shared between the two) which also linked to those respective tables using foreign keys. We decided on this design because of the efficiency and low cost of storing, reading, and writing Boolean values in a finite set of columns.

Approaching the end of final date of the project, our client requested a demo and a temporary deployment solution. Our solution to this was to use the temporary free tiers for Amazon RDS which made use of MySQL architecture, whereby we interfaced with the database through MySQL workbench's server SSH feature. However, since this was only a temporary solution, we had to revert our database solution back to localhost for final handoff.

Figure 4.3 Database schema



RMIT Classification: Trusted

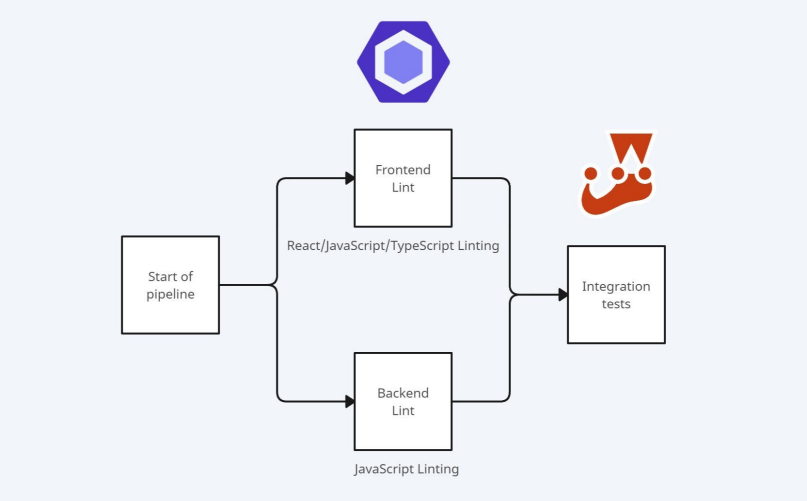
CONTINUOUS INTEGRATION PIPELINE

This was the planned architecture for our automated testing pipeline, using separate ESLint configurations for the backend and frontend parts of the codebase. The front-end linter using the ESLint React and TypeScript plugins to test for formatting errors, while the back-end lint simply took advantage of ESLint and its recommended JavaScript settings.

If both linting tests successfully pass, it will then move on to running the integration test suite with Jest. As our client did not give us a hosted DB to work with from the beginning of the project, we were not able to run integration tests on the pipeline. We could have done so when we deployed the app for testing, however, that would have been an inefficient use of time and effort, with there being no real reason to add it to the pipeline at that point.

The pipeline runs whenever there is a push or pull request, making sure to prevent any errors from making their way through merges into the development branch.

Figure 4.4 CI pipeline



5 TECHNICAL FRAMEWORK

The solution has been built using a combination of technologies, environments, and languages. Here's a breakdown:

TESTING:

- Jest: This is a JavaScript testing framework that is commonly used for testing JavaScript code, including Express APIs.
- GitHub Actions: It's a CI/CD (Continuous Integration/Continuous Deployment) platform provided by GitHub. It automates the testing processes of the codebase.
- ESLint: ESLint is a static code analysis tool for identifying and fixing problematic patterns found in JavaScript code. It helps maintain code quality and consistency.

BACKEND:

- Node.js: This is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside of a web browser. It is commonly used for building server-side applications.
- Express: Express.js is a web application framework for Node.js. It simplifies the process of building robust and scalable web applications by providing a simple and intuitive API.

FRONTEND:

- React: It is a JavaScript library that helps develop user interfaces for web applications more efficiently.
- Ionic: A framework for building cross-platform applications using web technologies like Angular, Vue and (what we used) React.

DATABASE:

- MySQL: MySQL is an open-source relational database management system (RDBMS) that uses structured query language (SQL) for managing and querying data. It's widely used in web development for its speed, reliability, and ease of use.

DEPLOYMENT SOLUTIONS:

- EC2: A web service provided by AWS (Amazon Web Services), that allows for the provision of virtualized computing resources in the cloud. Allowing us to create a virtual machine to run our applications.

RMIT Classification: Trusted

- RDS: Amazon Relational Database Service is a distribution relational database service by Amazon Web Services. Enabled a clean transition to migrate a relational database to the cloud.

VERSION CONTROL:

- GitHub: GitHub is a web-based platform that provides version control using Git. It offers various collaboration features, including issue tracking, pull requests, and more. It's a crucial tool for managing and sharing code among developers.

These technologies and tools collectively form the development and deployment environment for the solution. Node.js with Express serves as the backend framework, MySQL handles the database operations, Jest ensures proper testing, ESLint maintains code quality, and GitHub, along with GitHub Actions, supports version control and continuous integration workflows. This combination provides a robust and efficient foundation for building and maintaining the solution.

6 IMPLEMENTATION

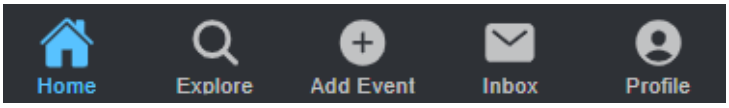
The implementation of the application has been done using the Ionic framework, with HTTP connections to the backend which is run through Node.JS Express and a MySQL database.

The application starts off with a login screen (see Figure 6.2), where the user enters their details. They can also sign up (see Figure 6.3). On sign up, they are greeted with a screen to select the type of events which they want to view (see Figure 6.4). Afterwards, they are put to the home page which shows a list of all the events which have been created thus far. They are then able to go to the profile page (see Figure 6.5), where they can press to edit their information, as well as the tags which were mentioned before.

Although we had more in mind for what the application was to do, we focused more on trying to capture a design that both was what the client wanted and was something that users would be able to enjoy. We also pushed further on the backend, where there are several API endpoints which were not utilised on the Frontend's side as per the submission.

Below is one picture of the original navbar (Figure 6.1) we had which had a lot of different pages, however most of the items on there were not implemented, so we decided to omit them from the pages of our final submission as to better show what had and hadn't been done so far.

Figure 6.1 Tag selection



RMIT Classification: Trusted

Figure 6.2 Log in page

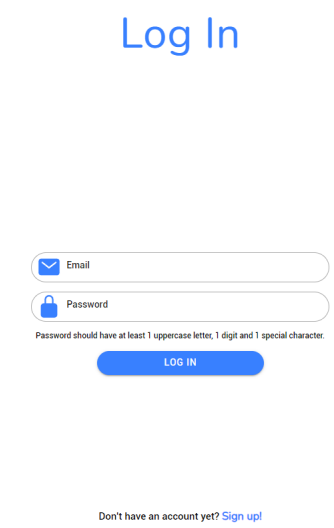


Figure 6.3 Log in page

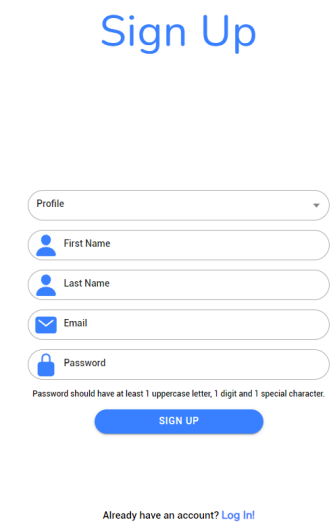


Figure 6.4 Tag selection

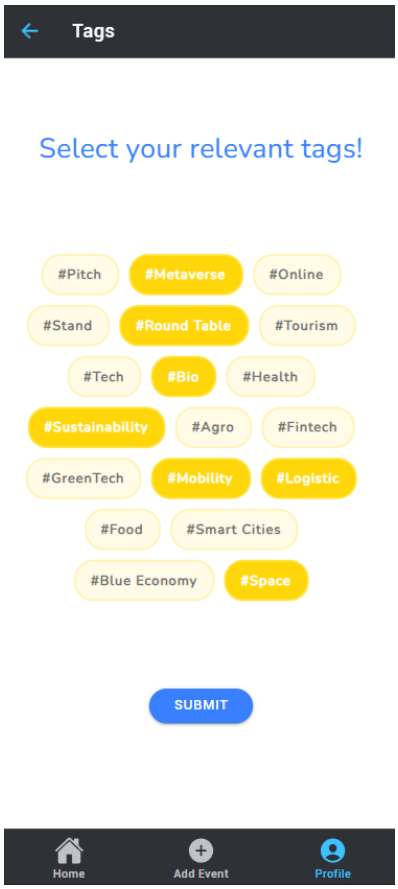
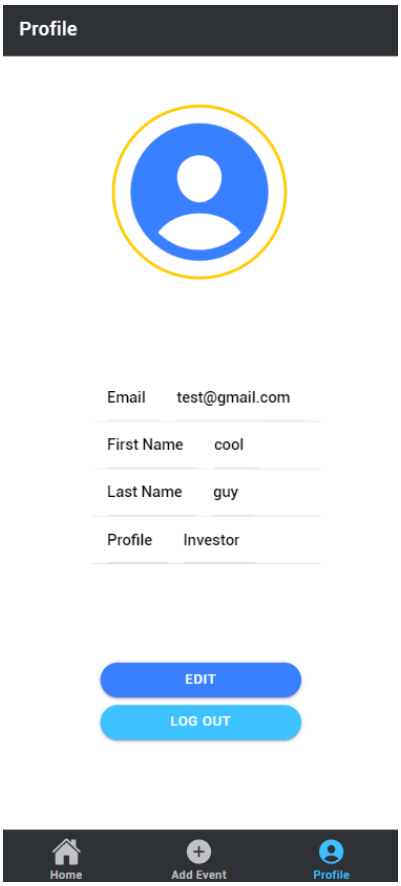


Figure 6.5 Profile page



7 DEPLOYMENT INSTRUCTIONS

The deployment of the application can be done on any cloud software, as long as it is able to have users connect to it. During the testing and demonstration for this project, the team used Amazon EC2 for the backend and frontend parts, and Amazon RDS for the database. The different parts were all on the same EC2 instance, however it is possible to also separate the two.
For the backend portion of the code, all that needs to be done is the following:

1. Upload the code onto the cloud
2. Install dependencies
3. Have the code be run
4. Set up automatic starting and restarting with the use of SystemCTL on Linux.
5. Make sure that the database being used is the one which has been set up

The database will automatically have the format given to it on the first run of the backend's code.

For the frontend portion of the code, the setting up is similar, but has a few extra steps. Perform the following:

1. Upload the code onto the cloud
2. Install dependencies
3. Build the Ionic application
4. Set up the code to be run on NGINX or similar
5. Have the code be run/start up with NGINX
6. Set up automatic starting and restarting with the use of SystemCTL on Linux.
7. Make sure that the API endpoint is pointing towards the correct URL

Note: for NGINX, the easiest way to set up the code is to create a config within the conf.d folder of NGINX (make sure the line "`include /etc/nginx/conf.d/*.conf;`" is included in the nginx.conf file) and include something similar to the following:

```
server {
  listen 80;
  root /home/ec2-user/VC-268-EventConnect/Frontend/dist;
  index index.html;

  access_log /var/log/nginx/innovent.com.access.log;
  error_log /var/log/nginx/innovent.com.error.log;
  location / {
    try_files $uri /index.html =404;
  }
}
```

Make sure to change the path if you have moved the file to somewhere else. There may also be some file permission errors, so there may be a need to use the chmod command to allow for the nginx user to be able to access the home directory, or wherever the application has been put.

8 TEST SPECIFICATIONS

Describe the testing frameworks you used within the project (if any).
Present the test cases related to the requirement specification, e.g., following the below template (use landscape formatting for the corresponding pages).

For this project we used the JavaScript testing framework, Jest.

Table 8.1 Testing

Test Case ID	Req. covered	Test Objective	Preconditions	Steps	Test data	Expected result
TC1	Req. 2	GET /api/Users/:id should return 404 if user ID does not exist	None	1. Send a GET request to `/api/Users/:id` with a non-existent user ID	Non-existing UserID	Response status code should be 404.
TC2	Req. 2	POST /api/Users/Register should return 422 if invalid email is provided	None	1. Send a POST request to `/api/Users/Register` with an invalid email.	Invalid email	Response status code should be 422.
TC3	Req. 2	POST /api/Users/Register should return 422 if invalid password is provided	None	1. Send a POST request to `/api/Users/Register` with an invalid password.	Invalid password	Response status code should be 422.
TC4	Req. 2	POST /api/Users/Register should return 422 if argument is missing	None	1. Send a POST request to `/api/Users/Register` with a missing password argument.	Missing password argument	Response status code should be 422.
TC5	Req. 2	POST /api/Users/Register should create a new user	None	1. Send a POST request to `/api/Users/Register` with valid user data.	Valid user data	Response status code should be 200, and a new user should be created.

TC6	Req. 3	GET /api/Users should return a list of users	None	1. Send a GET request to `/api/Users`	None	Response status code should be 200, and the response should contain a list of users.
TC7	Req. 3	GET /api/Users/:id should return a user by ID	A user with the specified ID exists	1. Send a GET request to `/api/Users/:id` with an existing user ID	Existing user ID	Response status code should be 200.
TC8	Req. 1	POST /api/Users/Login should return 200 for successful login	User with valid credentials exists	1. Send a POST request to `/api/Users/Login` with valid login credentials.	Valid login credentials	Response status code should be 200.
TC9	Req. 1	POST /api/Users/Login should return a 404 for unsuccessful login where username is correct	User with valid username but incorrect password exists	1. Send a POST request to `/api/Users/Login` with correct username and incorrect password.	Correct username, incorrect password	Response status code should be 404, and error message should indicate incorrect password.
TC10	Req. 1	POST /api/Users/Login should return a 404 for unsuccessful login	User with invalid email or username does not exist	1. Send a POST request to `/api/Users/Login` with invalid email or username.	Invalid email or username	Response status code should be 404, and error message should indicate user not found.
TC11	Req. 6, 7	PUT /api/Users/:id should update a user	User with the specified ID exists	1. Send a PUT request to `/api/Users/:id` with an existing user ID and updated data.	Existing user ID and updated data	Response status code should be 400.
TC12	Req. 12	GET /api/Users/Events/:id should return 404 if there are no events associated with user	User with no associated events exists	1. Send a GET request to `/api/Users/Events/:id` with a user ID having no associated events.	User ID with no associated events	Response status code should be 404.
TC13	Req. 6	DELETE /api/Users/:id should delete a user	User with the specified ID exists	1. Send a DELETE request to `/api/Users/:id` with an existing user ID.	Existing user ID	Response status code should be 200.
TC14	Req. 6	DELETE /api/Users/:id	User with the	1. Send a DELETE request to	Non-existent	Response status code

		should return 404 if the user is not found	specified ID does not exist	`/api/Users/:id` with a non-existent user ID.	user ID	should be 404.
TC15	Req. 3	GET /api/Events should return a list of events	An event exists	1. Send a GET request to `/api/Events`	None	Response status code should be 200, and the response should contain a list of events.
TC16	Req. 5	POST /api/Events should create a new event	None	1. Send a POST request to `/api/Events` with valid event data.	Valid event data	Response status code should be 201, and a new event should be created.
TC17	Req. 10	POST /api/Events/AddAttendee should enlist an attendee into an event	Event with the specified ID exists	1. Send a POST request to `/api/Events/AddAttendee/:id` with an existing event ID and user ID.	Existing event and user IDs	Response status code should be 201.
TC18	Req. 5	POST /api/Events should return a 409 when creating an event with a name that already exists	Event with the same name already exists	1. Send a POST request to `/api/Events` with an event name that already exists.	Existing event name	Response status code should be 409.
TC19	Req. 5	PUT /api/Events should return 200 if an event is updated successfully	Event with the specified ID exists	1. Send a PUT request to `/api/Events/:id` with an existing event ID and updated data.	Existing event ID and new update data	Response status code should be 200.
TC20	Req. 3	GET /api/Events/:id should return an event by its ID	Event with the specified ID exists	1. Send a GET request to `/api/Events/:id` with an existing event ID.	Existing event ID	Response status code should be 200.
TC21	Req. 5	DELETE /api/Events/:id should delete an event	Event with the specified ID exists	1. Send a DELETE request to `/api/Events/:id` with an existing event ID.	Existing event ID	Response status code should be 200.
TC22	Req. 3	POST /api/Events/AddAttendee should return 404 if event does not exist	Event with the specified ID does not exist	1. Send a POST request to `/api/Events/AddAttendee/:id` with a non-existent event ID.	Non-existent event ID	Response status code should be 404.

TC23	Req. 6	PUT api/Tags/UserInterests/:user_id should return 200 if user preferences are updated successfully	User with the specified ID exists	1. Send a PUT request to `/api/Tags/UserInterests/:user_id` with an existing user ID and updated preferences.	Existing user ID and updated preferences	Response status code should be 200.
TC24	Req. 6	PUT api/Tags/EventTags/:event_id should return 200 if event preferences are updated successfully	Event with the specified ID exists	1. Send a PUT request to `/api/Tags/EventTags/:event_id` with an existing event ID and updated preferences.	Existing event ID and updated preferences	Response status code should be 200.
TC25	Req. 3	GET api/Tags/UserByTag/:column/:bool should return 200 if a user with the tag exists	User with the specified tag exists	1. Send a GET request to `/api/Tags/UserByTag/:column/:bool` with an existing tag.	Existing tag	Response status code should be 200.
TC26	Req. 7	GET api/Tags/UserTagList/:user_id should return a user's tag list	User with the specified ID exists	1. Send a GET request to `/api/Tags/UserTagList/:user_id` with an existing user ID.	Existing user ID	Response status code should be 200.
TC27	Req. 3	GET api/Tags/EventTagList/:event_id should return an event's tag list	Event with the specified ID exists	1. Send a GET request to `/api/Tags/EventTagList/:event_id` with an existing event ID.	Existing event ID	Response status code should be 200.
TC28	Req. 8	POST /api/Enlist/ should return 200 if a user is successfully enlisted to an event	User and event with the specified IDs exist	1. Send a POST request to `/api/Enlist/` with existing user and event IDs.	Existing user and event IDs	Response status code should be 200.
TC29	Req. 3	GET api/Enlist/Events/:user_id should return a list of events the user is attending	User with the specified ID exists	1. Send a GET request to `/api/Enlist/Events/:user_id` with an existing user ID.	Existing user ID	Response status code should be 200.

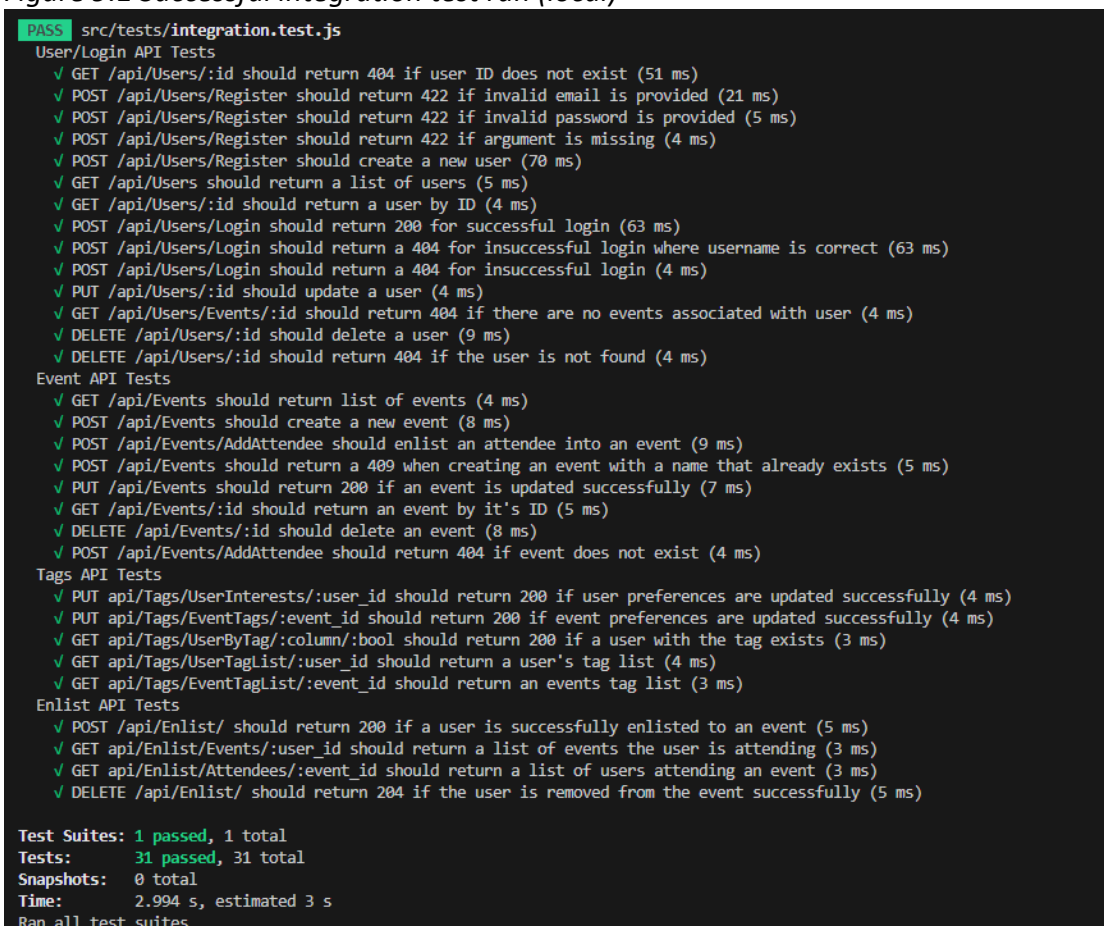
TC30	Req. 3	GET api/Enlist/Attendees/:event_id should return a list of users attending an event	Event with the specified ID exists	1. Send a GET request to `/api/Enlist/Attendees/:event_id` with an existing event ID.	Existing event ID	Response status code should be 200.
TC31	Req. 3	DELETE /api/Enlist/ should return 204 if the user is removed from the event successfully	User and event with the specified IDs exist	1. Send a DELETE request to `/api/Enlist/` with existing user and event IDs.	Existing user and event IDs	Response status code should be 204.

9 TESTING RESULTS

Testing results were successful, although during creating the tests there were a few inconsistencies with status code and Json responses found, however, the process of making the tests created unobstructed vision of what issues needed to be corrected.

As a result of test implementation, adding new features whilst not breaking or messing with previous functionality became much simpler to ensure. It also made the process of debugging and creating new backend features easier and significantly more efficient.

Figure 9.1 Successful integration test run (local)



```
PASS src/tests/integration.test.js
  User/Login API Tests
    √ GET /api/Users/:id should return 404 if user ID does not exist (51 ms)
    √ POST /api/Users/Register should return 422 if invalid email is provided (21 ms)
    √ POST /api/Users/Register should return 422 if invalid password is provided (5 ms)
    √ POST /api/Users/Register should return 422 if argument is missing (4 ms)
    √ POST /api/Users/Register should create a new user (70 ms)
    √ GET /api/Users should return a list of users (5 ms)
    √ GET /api/Users/:id should return a user by ID (4 ms)
    √ POST /api/Users/Login should return 200 for successful login (63 ms)
    √ POST /api/Users/Login should return a 404 for insuccessful login where username is correct (63 ms)
    √ POST /api/Users/Login should return a 404 for insuccessful login (4 ms)
    √ PUT /api/Users/:id should update a user (4 ms)
    √ GET /api/Users/Events/:id should return 404 if there are no events associated with user (4 ms)
    √ DELETE /api/Users/:id should delete a user (9 ms)
    √ DELETE /api/Users/:id should return 404 if the user is not found (4 ms)
  Event API Tests
    √ GET /api/Events should return list of events (4 ms)
    √ POST /api/Events should create a new event (8 ms)
    √ POST /api/Events/AddAttendee should enlist an attendee into an event (9 ms)
    √ POST /api/Events should return a 409 when creating an event with a name that already exists (5 ms)
    √ PUT /api/Events should return 200 if an event is updated successfully (7 ms)
    √ GET /api/Events/:id should return an event by it's ID (5 ms)
    √ DELETE /api/Events/:id should delete an event (8 ms)
    √ POST /api/Events/AddAttendee should return 404 if event does not exist (4 ms)
  Tags API Tests
    √ PUT api/Tags/UserInterests/:user_id should return 200 if user preferences are updated successfully (4 ms)
    √ PUT api/Tags/EventTags/:event_id should return 200 if event preferences are updated successfully (4 ms)
    √ GET api/Tags/UserByTag/:column/:bool should return 200 if a user with the tag exists (3 ms)
    √ GET api/Tags/UserTagList/:user_id should return a user's tag list (4 ms)
    √ GET api/Tags/EventTagList/:event_id should return an events tag list (3 ms)
  Enlist API Tests
    √ POST /api/Enlist/ should return 200 if a user is successfully enlisted to an event (5 ms)
    √ GET api/Enlist/Events/:user_id should return a list of events the user is attending (3 ms)
    √ GET api/Enlist/Attendees/:event_id should return a list of users attending an event (3 ms)
    √ DELETE /api/Enlist/ should return 204 if the user is removed from the event successfully (5 ms)

Test Suites: 1 passed, 1 total
Tests:       31 passed, 31 total
Snapshots:   0 total
Time:        2.994 s, estimated 3 s
Ran all test suites
```

10CYBER SECURITY

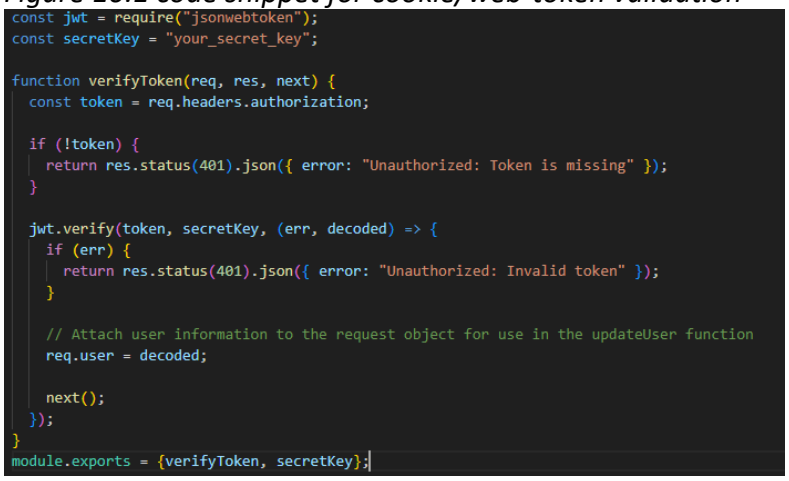
In our project, upholding principles of cybersecurity was a great concern to ensure the confidentiality, integrity, and availability of sensitive data and information. We have taken several important measures to safeguard against potential cyber threats and vulnerabilities.

VALIDATION

One of the key aspects of cybersecurity that we have addressed is function validation. By carefully validating and authorizing every function, we ensure that only authorized users can interact with specific features, thereby reducing the risk of exploitation by malicious users. This is essential for maintaining the overall security of our project.

Token and model validation also play a critical role in our cybersecurity strategy. We implemented validation checks both in the front and backend to ensure data sanitization. By verifying the authenticity and integrity of data inputs, we can prevent injection attacks, such as SQL injection or cross-site scripting (XSS). This validation process significantly reduces the risk of data manipulation and unauthorized access, which are common vectors for cyberattacks We used libraries like ‘jsonwebtokens’ and ‘joi’ to implement these.

Figure 10.1 code snippet for cookie/web-token validation



```
const jwt = require("jsonwebtoken");
const secretKey = "your_secret_key";

function verifyToken(req, res, next) {
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).json({ error: "Unauthorized: Token is missing" });
  }

  jwt.verify(token, secretKey, (err, decoded) => {
    if (err) {
      return res.status(401).json({ error: "Unauthorized: Invalid token" });
    }

    // Attach user information to the request object for use in the updateUser function
    req.user = decoded;

    next();
  });
}
module.exports = {verifyToken, secretKey};
```

Figure 10.2 code snippet for data validation that is consistent across front and backend



```
const Joi = require('joi');
const subCategories = require('../eventTags.js');

// Define a Joi schema for the User model
const userSchema = Joi.object({
    first_name: Joi.string().min(3).max(20).required(),
    last_name: Joi.string().min(3).max(20).required(),
    photo: Joi.string(),
    email: Joi.string().email().required(),
    password: Joi.string().pattern(new RegExp('^(?=.*[A-Z])(?=.*\\d)(?=.*[\\W_]).{7,}$')).required().max(20),
    profile: Joi.string()
        .valid(...subCategories.profile.tags)
        .required(),
});
const updateUserSchema = Joi.object({
    first_name: Joi.string().min(3).max(20),
    last_name: Joi.string().min(3).max(20),
    photo: Joi.string(),
    email: Joi.string().email(),
    password: Joi.string().pattern(new RegExp('^(?=.*[A-Z])(?=.*\\d)(?=.*[\\W_]).{7,}$')).max(20),
    profile: Joi.string().valid(...subCategories.profile.tags),
});

module.exports = {
    userSchema,
    validateUser: (user) => userSchema.validate(user),
    validateUpdateUser: (user) => updateUserSchema.validate(user),
};
```

PASSWORD HASHING

Hashing passwords is an essential step to protect user credentials. Storing passwords in hashed form ensures that even if the database is compromised, the actual passwords remain secure. This makes it significantly harder for attackers to access user accounts, as they cannot reverse-engineer the hashed passwords.

Figure 10.3 example of password hashing

password
$2b$10$KwqH6aR6qcKfcIOvnJemmeJ04CC8UqvIxS4W2e7nbdw...
$2b$10$gcTz4fqEqUIwTw9sbpZr0.mf7pZT.nWcFzZOgjybfvC...
$2b$10$GypBfkvSjE.zJpmVwZr8VOFfOzFfZMooZbyE0v8ng4o...

SECURING ENDPOINTS AND CLOUD DEPLOYMENT

Securing endpoints is another vital cybersecurity measure. We have implemented stringent security controls to protect the endpoints through which our application communicates with users and other systems. This includes the use of encryption protocols like HTTPS to encrypt data in transit, as well as robust authentication and authorization mechanisms to verify the identity of users and limit access to authorized parties only. We accomplished this when we had to temporarily migrate our backend and frontend to the cloud.

Hosting our database on Amazon RDS (for demoing our project in a foreign country) adds an additional layer of cybersecurity. Amazon RDS provides a secure, managed database environment with features like automated backups, encryption, and access control. This ensures that our data is stored and managed in a way that minimizes the risk of data breaches and unauthorized access.

RMIT Classification: Trusted

11 IMPACT ON STAKEHOLDERS

Innovent's central mission revolves around offering an array of event options that cater specifically to the preferences and interests of its stakeholders and users. Through a robust verification process, individuals and organisations gain the authority to manage their own events, affording them the flexibility to designate these gatherings as either private, for more intimate affairs, or public, to encourage broader participation and a heightened sense of community ownership.

Envisioning a future of global accessibility, Innovent aspires to serve as a hub that unites individuals from every corner of the globe. By doing so, it seeks to act as a catalyst for vibrant cultural exchange and sustained connectivity. This initiative stands to deliver far-reaching benefits, not only by fostering increased social interaction and a deeper appreciation of local and international events, but also by championing the principles of diversity and inclusivity.